VIETNAM UNIVERSITY-INTERNATIONAL UNIVERSITY (VNU-HCMIU)

COMPUTER SCIENCE AND INFORMATION DEPARTMENT

FINAL REPORT: WEB APPLICATION DEVELOPMENT

FURNITUREE

Furniture E-Commerce Web Application

Final Report

Field	Information
Course	Web Application Development
Instructor	Mr. Nguyen Trung Nghia
Team size	2 members
Team member 1	Tran Le Minh Quan - Student ID: **ITITWE23042**
Team member 2	Nguyen Phuoc Thinh - Student ID: **ITITWE22106**
Live demo URL	Furnituree
GitHub repository	https://github.com/Edway108/E-commerce-web-page-for-Furniture-Sale.git https://github.com/PhuocThinh09/E-commerce-web-page-for-Furniture-Sale (for check the number of commits)

Contents

1. Executive Summary

Furnituree is a full-stack e-commerce web application designed for browsing, managing, and purchasing furniture products. The system focuses on a realistic online furniture store scenario where customers can explore products, search by category, add products to cart, complete checkout, and communicate with administrators through real-time chat. Administrators and managers can manage products, categories, users, orders, reviews, and operational data through a protected dashboard.

The project was developed for the Web Application Development course and demonstrates full-stack integration between a responsive frontend, a Spring Boot backend, and a MySQL database. The implementation follows layered architecture principles, RESTful API design, server-side validation, authentication, authorization, business logic, error handling, and basic deployment support using Docker.

- Project name: Furnituree - Used Furniture E-Commerce Web Application.
- Main users: customers, administrators, managers, and guest visitors.
- Core modules: authentication, product catalog, cart, checkout, orders, admin management, search, and chat.
- Advanced features: file upload, CSV export, dashboard analytics, real-time chat with admin notifications, Docker-based deployment support.
- Technology stack: HTML, CSS, JavaScript, Spring Boot, Spring Security, Spring Data JPA, Hibernate, MySQL, WebSocket/STOMP, Docker.

Area	Summary
Frontend	Static HTML/CSS/JavaScript served from Spring Boot resources. Responsive layout with modern blue-themed UI and customer/admin pages.
Backend	Spring Boot REST API with layered MVC/service/repository design, JWT authentication, RBAC, validation, and centralized error handling.
Database	MySQL schema with related entities including users, products, categories, tags, cart, order, payment, review, wishlist, and audit logs.
Security	BCrypt password hashing, JWT-based authentication, role-based access control, CORS configuration, and protected admin endpoints.
Deployment	Supports local Maven execution and Docker Compose execution with application and MySQL containers.

2. Requirements Analysis

2.1 Problem Statement

Furniture shopping often involves comparing product types, prices, availability, and delivery options. A small furniture business needs a simple but professional web application to display products, support customer orders, and manage administrative operations. Without a centralized system, product management, customer communication, order tracking, and inventory control become inefficient and error-prone.

Furnituree solves this problem by providing a web-based system where customers can browse furniture products and administrators can manage products, categories, users, orders, and customer support conversations from one application.

2.2 Target Users and Pain Points

User Type	Pain Points	System Support
Guest visitor	Needs to browse products quickly before registering.	Public product catalog, categories, search, product details.
Customer	Needs a smooth shopping experience and easy checkout.	Register, login, cart, checkout, order history, chat support.
Admin	Needs to manage products, users, and system data.	Admin dashboard, product CRUD, user management, review/order management.
Manager	Needs business overview and operational control.	Dashboard summary, product/order monitoring, reports, search tools.

2.3 User Stories

Priority	User Story
Must	As a guest, I want to view furniture products so that I can decide whether to register.
Must	As a customer, I want to register and login securely so that I can access cart and checkout features.
Must	As a customer, I want to search products by keyword so that I can find items faster.
Must	As a customer, I want to filter products by category so that I can browse relevant furniture types.
Must	As a customer, I want to add products to my cart so that I can purchase multiple items together.
Must	As a customer, I want to remove individual items from my cart so that I can correct my order before checkout.
Must	As a customer, I want to checkout with shipping and payment information so that I can place an order.
Must	As an admin, I want to login with an admin role so that I can access management features.
Must	As an admin, I want to create, update, and delete products so that the catalog stays accurate.
Must	As an admin, I want to search products in the admin page so that I can manage large catalogs faster.
Should	As an admin, I want to manage users and roles so that system access is controlled.
Should	As a manager, I want dashboard statistics so that I can monitor business performance.
Should	As a customer, I want real-time chat support so that I can ask questions before buying.
Should	As an admin, I want chat notifications so that I know when a customer needs support.
Should	As a customer, I want to review products so that other users can evaluate item quality.
Nice to Have	As an admin, I want CSV export so that I can analyze data outside the system.
Nice to Have	As a developer, I want Docker support so that the system can be deployed consistently.
Nice to Have	As a user, I want responsive pages so that I can use the application on laptop, tablet, or mobile.

2.4 Functional Requirements

- The system shall allow users to register, login, logout, view profile, and change password.
- The system shall support at least three roles: ADMIN, MANAGER, and CUSTOMER.
- The system shall enforce role-based access control for sensitive endpoints and admin pages.
- The system shall provide CRUD operations for products, categories, users, orders, and related domain entities.
- The system shall provide product search, category filtering, sorting, and pagination support.
- The system shall allow customers to add products to cart, remove individual items, and proceed to checkout.
- The system shall validate inventory before checkout and prevent overselling.
- The system shall create orders with order items, payment information, delivery information, and status tracking.
- The system shall support file upload for product images with validation.
- The system shall provide dashboard analytics, CSV export, and real-time customer-admin chat.

2.5 Non-Functional Requirements

Category	Requirement
Security	Passwords must be stored using BCrypt hashing. Protected endpoints must require JWT tokens and role validation.
Performance	Common catalog and admin requests should respond within approximately 2 seconds under demo data volume.
Usability	The UI must be clean, responsive, and understandable for non-technical users.
Reliability	The application should fail gracefully with user-friendly error messages instead of raw stack traces.
Maintainability	The codebase should separate controller, service/business logic, repository, model, DTO, and configuration layers.
Portability	The project should run locally through Maven and optionally through Docker Compose.

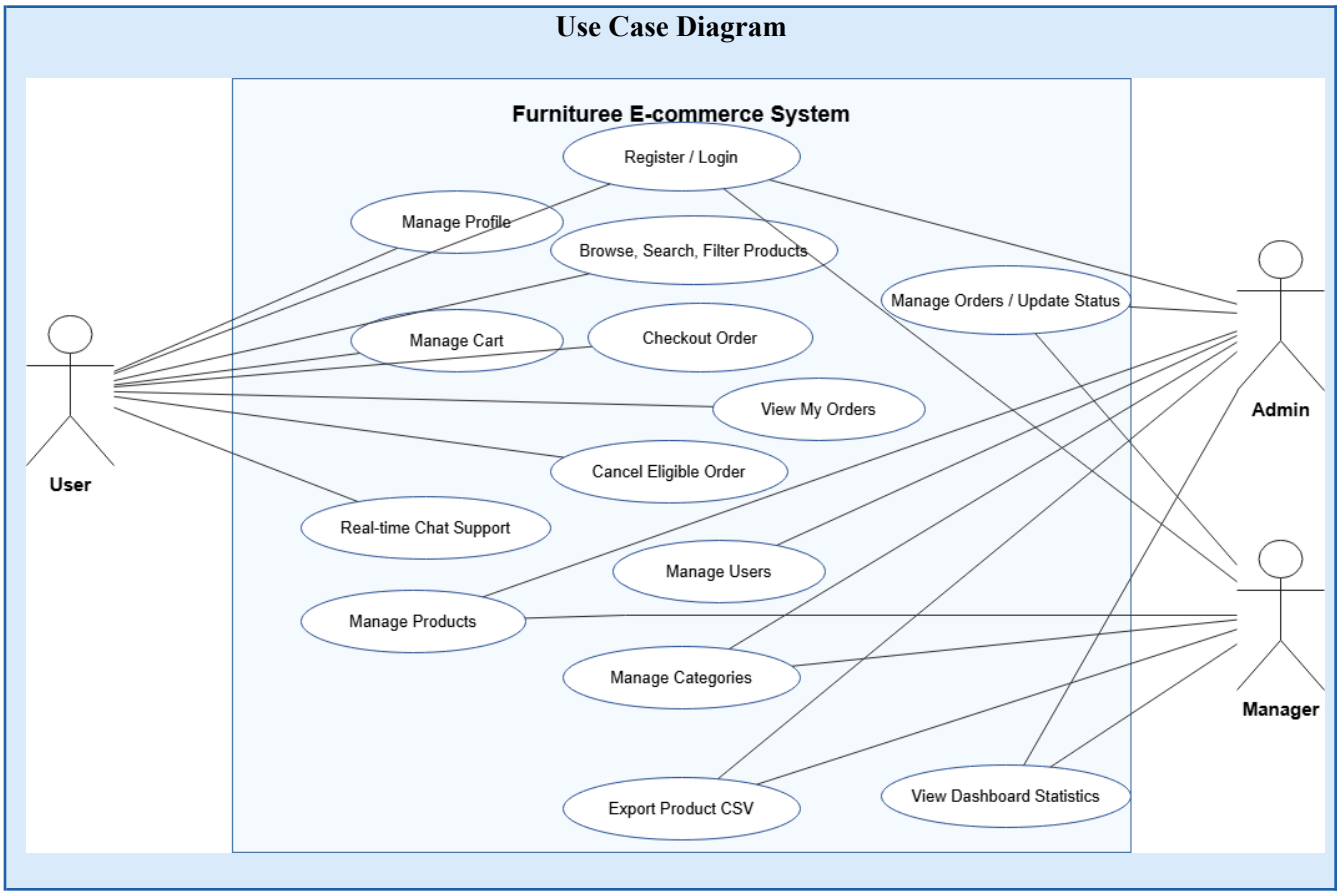
3. Use Case Analysis

3.1 Actors

Actor	Description
Customer	A registered user who can manage cart, checkout, review products, and chat with admin.
Admin	A privileged user who can manage products, users, orders, reviews, reports, and chat support.

Manager	A privileged user who can monitor dashboard and business operations.

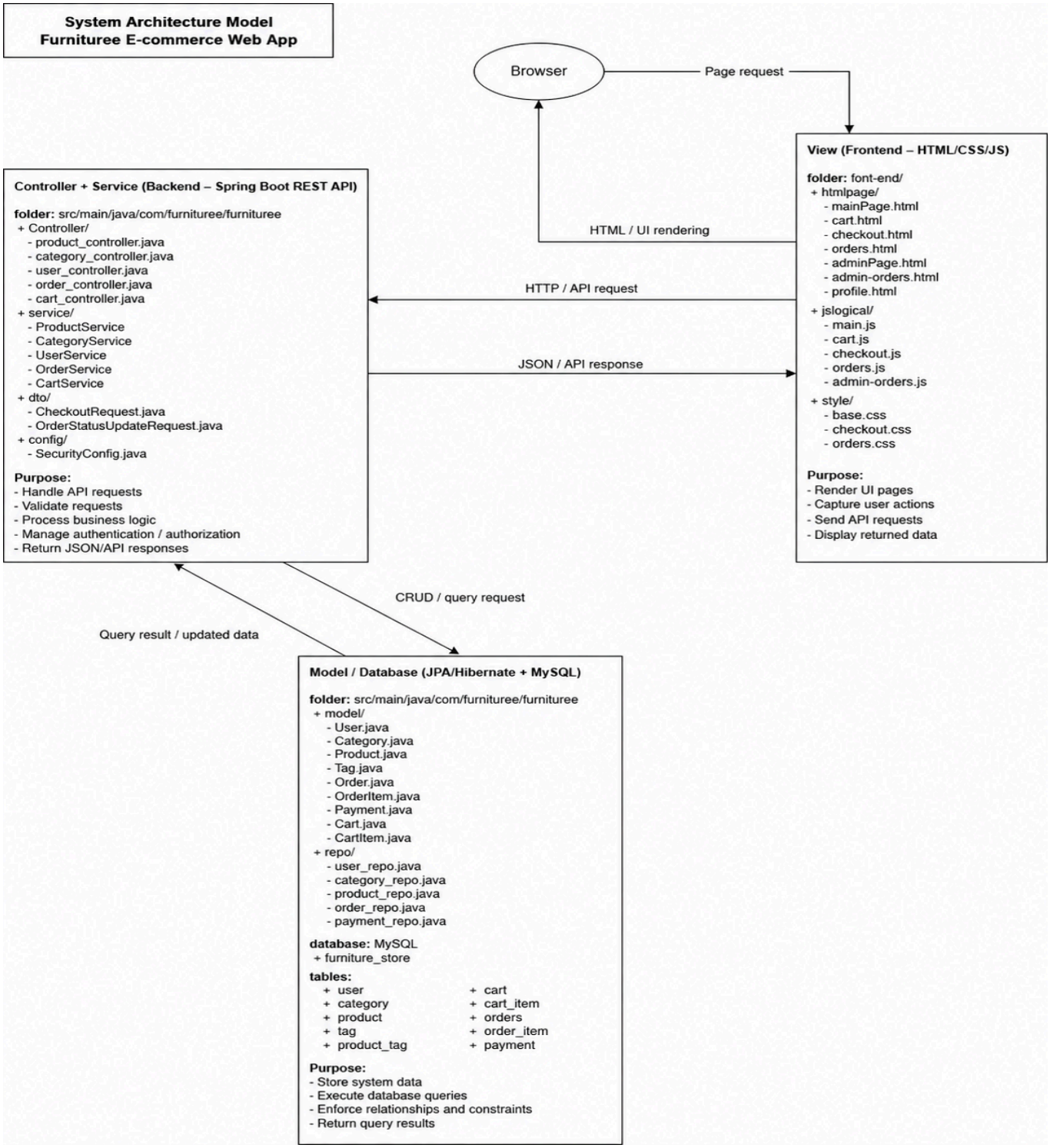
3.2 Use Cases diagram



4. System Architecture

4.1 Architecture Overview

Furnituree uses a client-server architecture. The frontend is served as static assets and communicates with backend REST endpoints using HTTP requests. The backend is implemented with Spring Boot and follows a layered design. MySQL stores persistent data, while WebSocket/STOMP is used for real-time chat and admin notifications.



4.1.2 Component Explanation

1. Client Side / Frontend

The frontend is implemented using **HTML, CSS, and JavaScript**. It provides the user interface for customers, admins, and managers.

The main frontend responsibilities include:

- Displaying products and product details
- Providing search, filter, sort, and pagination features
- Managing the shopping cart interface
- Collecting checkout information
- Displaying user order history
- Displaying admin dashboard and management pages
- Sending requests to the backend REST API
- Storing and sending JWT token for authenticated requests

The frontend communicates with the backend using JavaScript `fetch()` requests. For protected features, the frontend sends the JWT token in the request header.

2. Backend / Spring Boot Server

The backend is built with **Spring Boot**. It exposes REST API endpoints and contains the main business logic of the system.

The backend follows a layered architecture:

Controller Layer

→ Service Layer

→ Repository Layer

→ Database

The **Controller Layer** receives HTTP requests from the frontend and returns HTTP responses. It should not contain complex business logic.

The **Service Layer** contains the main business logic, such as checkout processing, stock validation, payment creation, order status updates, and user validation.

The **Repository Layer** uses Spring Data JPA to communicate with the MySQL database.

3. Security Component

The system uses **Spring Security and JWT authentication**.

When a user logs in successfully, the backend generates a JWT token. The frontend stores this token and sends it with future requests.

The main roles are:

user → Browse products, manage cart, checkout, view orders

manager → Manage products, categories, orders, dashboard

admin → Full management access, including user management

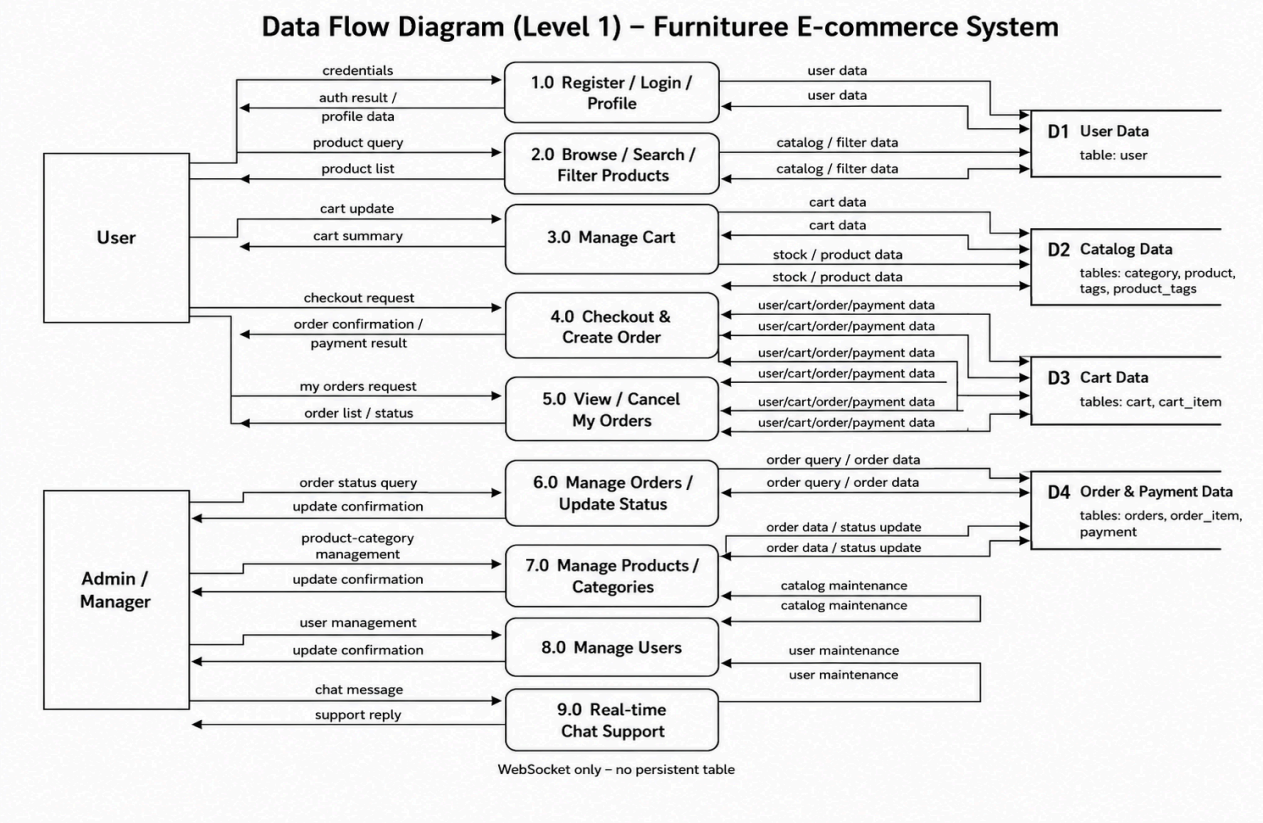
4. Database / MySQL

The database stores all persistent data for the system. MySQL is used as the relational database management system.

The database includes 10 entities

The database is designed to preserve important business history. For example, users and orders are not physically deleted in normal workflows. Instead, the system uses fields such as `active`, `status`, and `order_status`.

Data flow diagram



4.2 Technology Stack Justification

Layer	Selected Technology	Reason
Frontend	HTML, CSS, JavaScript	Simple, transparent, and suitable for demonstrating web fundamentals without framework complexity.
Backend	Spring Boot 3	Provides rapid REST API development, embedded server, validation, security integration, and industry-standard structure.
Security	Spring Security + JWT + BCrypt	Supports stateless authentication, password hashing, authorization, and protected endpoints.
Persistence	Spring Data JPA + Hibernate	Reduces boilerplate SQL, supports entity relationships, and uses parameterized queries to reduce SQL injection risk.
Database	MySQL 8.X	Widely used relational database suitable for structured e-commerce data and complex relationships.
Real-Time	WebSocket/STOMP	Allows two-way chat messages and live admin notifications.

Deployment	Docker Compose	Packages app and database into repeatable containers for demo and deployment.

4.3 Layered Structure

- Controller layer: receives HTTP/WebSocket requests and returns API responses.
- DTO layer: transfers request and response data without exposing internal entity details unnecessarily.
- Service layer: contains business rules such as cart updates, checkout validation, and order workflow.
- Repository layer: provides database access through Spring Data JPA repositories.
- Model/entity layer: represents relational database tables and relationships.
- Configuration layer: contains security, CORS, JWT, WebSocket, and application settings.

4.4 Design Patterns

Pattern	Application in Project
MVC	Controllers handle requests, models represent data, and frontend pages render the user interface.
Repository Pattern	Spring Data repositories isolate database operations from controllers and business logic.
DTO Pattern	Request and response DTOs separate API contracts from entity classes.
Dependency Injection	Spring injects repositories, services, and configuration beans automatically.
Singleton Beans	Spring-managed services/configuration objects are reused by the application context.

4.5 API Design

The Furnitureee system uses REST API to connect the HTML/CSS/JavaScript frontend with the Spring Boot backend. The frontend sends HTTP requests, and the backend returns JSON responses. Protected APIs require JWT authentication.

RESTful Principles

The API is designed around resources such as users, products, categories, cart, orders, payments, and tags. Each resource is accessed through clear URLs and standard HTTP methods.

GET → retrieve data

POST → create new data

PUT → update existing data

DELETE → delete, deactivate, or remove data

URL Structure

The API follows a resource-based URL structure:

/resource

/resource/{id}

/resource/action

Examples:

/auth/login

/auth/register

/products

/products/{id}

HTTP Methods Usage

Method	Purpose	Example
GET	Retrieve data	GET /products
POST	Create data	POST /orders/checkout
PUT	Update data	PUT /orders/{id}/status
DELETE	Remove/deactivate data	DELETE /products/{id}

The system uses `GET` for product lists and order history, `POST` for login, checkout, and creating records, `PUT` for updating records such as order status, and `DELETE` for removing or deactivating records.

Status Codes Strategy

Status Code	Meaning
200 OK	Request successful
201 Created	New resource created
400 Bad Request	Invalid input
401 Unauthorized	Missing or invalid JWT
403 Forbidden	User does not have permission
404 Not Found	Resource not found
409 Conflict	Duplicate or conflicting data
500 Internal Server Error	Unexpected server error

Authentication

The project uses JWT authentication. After login, the backend returns a token. The frontend sends this token in protected requests.

Authorization: Bearer <JWT_TOKEN>

Roles are used to control access:

user → cart, checkout, my orders

manager → product/category/order management

admin → full management access, including user management

Key Request and Response Examples

Login
POST /auth/login

Request:

```
{  "username": "admin",
  "password": "123456"
}
```

Response:

```
{  "token": "jwt-token-value",
  "username": "admin",
  "role": "admin"
}
```

Checkout
POST /orders/checkout

Request:

```
{  "fullName": "Nguyen Van A",
  "phoneNumber": "0987654321",
  "address": "123 Nguyen Trai Street",
  "city": "Ho Chi Minh City",
  "paymentMethod": "COD"
}
```

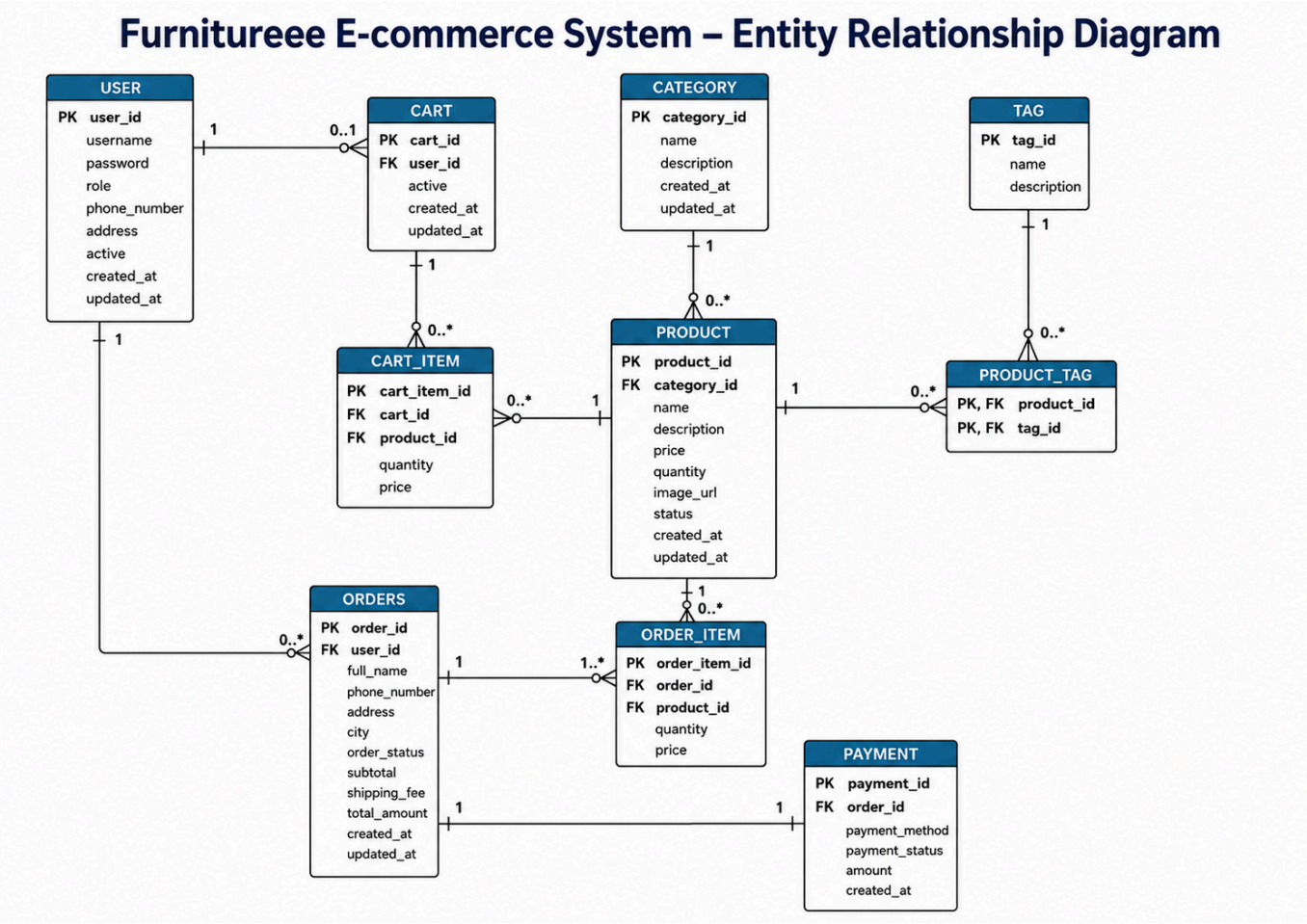
Response:

```
{  "message": "Checkout successful",
  "orderId": 15,
  "orderStatus": "PENDING",
  "paymentStatus": "UNPAID",
  "totalAmount": 1250.0
}
```

5. Database Design

5.1 Database Overview

The database is designed as a relational schema with normalized entities for users, products, categories, tags, cart, order, payment. The structure supports realistic e-commerce workflows and demonstrates one-to-many and many-to-many relationships.



5.2 Table Schemas

This section describes the database tables used in the Furnitureee E-commerce system. Each table includes its main columns, data types, primary keys, foreign keys, and important constraints.

Table: `user`

Stores account information for customers, admins, and managers.

- user_id: BIGINT PRIMARY KEY AUTO_INCREMENT

- username: VARCHAR(255) UNIQUE NOT NULL

- password: VARCHAR(255) NOT NULL

- role: VARCHAR(50) NOT NULL

- phonenumber: INT

- address: VARCHAR(255)

- active: BOOLEAN DEFAULT TRUE

- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

Constraints:

- username must be unique.

- role is used for authorization: admin, manager, user.

- active is used for soft delete/deactivation.

Table: `category`

Stores product categories such as Sofa, Table, Chair, Bed, Storage, Lighting, Office, Outdoor, Decor, and Dining.

- category_id: BIGINT PRIMARY KEY AUTO_INCREMENT

- name: VARCHAR(255) UNIQUE NOT NULL

- description: VARCHAR(1000)

- image_url: VARCHAR(500)

- status: VARCHAR(20) DEFAULT 'ACTIVE'

- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

Constraints:

- category name should be unique.

- status is used to mark categories as ACTIVE or INACTIVE.

Table: `product`

Stores furniture product information.

- product_id: BIGINT PRIMARY KEY AUTO_INCREMENT

- category_id: BIGINT FOREIGN KEY REFERENCES category(category_id)

- product_name: VARCHAR(120) UNIQUE NOT NULL

- description: VARCHAR(1000)

- price: DOUBLE NOT NULL

- quantity: BIGINT NOT NULL

- img: VARCHAR(500)

- status: VARCHAR(20) DEFAULT 'ACTIVE'

- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

Constraints:

- product_name must be unique.

- category_id must reference an existing category.

- quantity stores available stock.

- price must be greater than or equal to 0.

Table: `cart`

Stores the shopping cart of a user.

- cart_id: BIGINT PRIMARY KEY AUTO_INCREMENT

- user_id: BIGINT FOREIGN KEY REFERENCES user(user_id)

- active: BOOLEAN DEFAULT TRUE

- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

Constraints:

- user_id must reference an existing user.

- One user should have at most one active cart.

Table: `cart_item`

Stores products added to a cart.

- cart_item_id: BIGINT PRIMARY KEY AUTO_INCREMENT

- cart_id: BIGINT FOREIGN KEY REFERENCES cart(cart_id)

- product_id: BIGINT FOREIGN KEY REFERENCES product(product_id)

- quantity: INT NOT NULL

- price: DOUBLE NOT NULL

Constraints:

- cart_id must reference an existing cart.

- product_id must reference an existing product.

- quantity must be greater than 0.

- price stores the product price when the item is added to cart.

Table: `orders`

Stores customer order information after checkout.

- order_id: BIGINT PRIMARY KEY AUTO_INCREMENT

- user_id: BIGINT FOREIGN KEY REFERENCES user(user_id)

- full_name: VARCHAR(255) NOT NULL

- phone_number: VARCHAR(20) NOT NULL

- address: VARCHAR(255) NOT NULL

- city: VARCHAR(100) NOT NULL

- order_status: VARCHAR(30) NOT NULL DEFAULT 'PENDING'

- subtotal: DOUBLE NOT NULL

- shipping_fee: DOUBLE NOT NULL

- total_amount: DOUBLE NOT NULL

- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

Constraints:

- user_id must reference an existing user.

- order_status values include PENDING, CONFIRMED, SHIPPING, COMPLETED, CANCELLED.

- subtotal, shipping_fee, and total_amount are calculated during checkout.

- Orders are not hard deleted; cancellation is handled using order_status.

Table: `order_item`

Stores the products inside each order.

- order_item_id: BIGINT PRIMARY KEY AUTO_INCREMENT

- order_id: BIGINT FOREIGN KEY REFERENCES orders(order_id)

- product_id: BIGINT FOREIGN KEY REFERENCES product(product_id)

- quantity: INT NOT NULL

- price: DOUBLE NOT NULL

Constraints:

- order_id must reference an existing order.

- product_id must reference an existing product.

- quantity must be greater than 0.

- price stores the product price at checkout time.

Table: `payment`

Stores payment information for each order.

- payment_id: BIGINT PRIMARY KEY AUTO_INCREMENT

- order_id: BIGINT UNIQUE FOREIGN KEY REFERENCES orders(order_id)

- payment_method: VARCHAR(30) NOT NULL

- payment_status: VARCHAR(30) NOT NULL

- amount: DOUBLE NOT NULL

- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Constraints:

- order_id must reference an existing order.

- order_id should be unique because one order has one payment.

- payment_method values include COD, BANK_TRANSFER, MOCK_CARD.

- payment_status values include UNPAID, PAID, FAILED.

Table: `tags`

Stores product tags used for product classification and many-to-many relationship demonstration.

- tag_id: BIGINT PRIMARY KEY AUTO_INCREMENT

- name: VARCHAR(80) UNIQUE NOT NULL

- description: VARCHAR(255)

- status: VARCHAR(20) DEFAULT 'ACTIVE'

- created_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP

- updated_at: TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

Constraints:

- tag name must be unique.

- status is used to activate or deactivate a tag.

Table: `product_tags`

Junction table that resolves the many-to-many relationship between products and tags.

- product_id: BIGINT PRIMARY KEY, FOREIGN KEY REFERENCES product(product_id)

- tag_id: BIGINT PRIMARY KEY, FOREIGN KEY REFERENCES tags(tag_id)

Constraints:

- Composite primary key: product_id + tag_id.

- product_id must reference an existing product.

- tag_id must reference an existing tag.

- Prevents duplicate tag assignment for the same product.

5.3 Relationships explanation

The Furnitureee database is designed using relational database principles. Most relationships are one-to-many because one main record can own many related detail records. The many-to-many relationship between products and tags is resolved using a junction table.

User – Cart

USER 1 — 0..1 CART

One user can have zero or one active cart. Each cart belongs to one user. This supports the shopping cart feature for logged-in users.

Category – Product

CATEGORY 1 — 0..* PRODUCT

One category can contain many products, but each product belongs to one category. This supports category-based product filtering.

Product – CartItem

PRODUCT 1 — 0..* CART_ITEM

One product can appear in many cart items from different users. Each cart item refers to one product.

Product – OrderItem

PRODUCT 1 — 0..* ORDER_ITEM

One product can appear in many order items. This allows the system to track which products were purchased.

Product – Tags

User – Orders

USER 1 — 0..* ORDERS

One user can place many orders, but each order belongs to one user. This supports order history for each customer.

Cart – CartItem

CART 1 — 0..* CART_ITEM

One cart can contain many cart items. Each cart item belongs to one cart and stores product quantity and price.

Orders – OrderItem

ORDERS 1 — 1..* ORDER_ITEM

One order must contain at least one order item. Each order item belongs to one order and stores purchased product details.

Orders – Payment

ORDERS 1 — 1 PAYMENT

Each order has one payment record. Payment is separated into its own table because it has specific fields such as payment method, payment status, and amount.

PRODUCT * — * TAGS

Products and tags have a many-to-many relationship. One product can have many tags, and one tag can be assigned to many products.

This cannot be stored directly using only one foreign key, so the system uses a junction table.

ProductTags Junction Table

PRODUCT 1 — 0..* PRODUCT_TAGS

TAGS 1 — 0..* PRODUCT_TAGS

The `product_tags` table resolves the many-to-many relationship between `product` and `tags`.

It stores:

product_id

tag_id

This design avoids repeated columns such as `tag1`, `tag2`, `tag3` in the product table and keeps the database normalized.

Cascading Rules

The system avoids hard deleting important business records such as users, products, orders, and payments because they are part of transaction history.

Instead, the project uses status-based handling:

User deletion → deactivate user

Product unavailable → update stock/status

Order deletion → update order_status to CANCELLED

Payment issue → update payment_status

Cart after checkout → clear cart items

This protects referential integrity and prevents broken foreign key relationships while still supporting realistic e-commerce workflows.

5.4 Normalization Analysis

The database is designed to meet at least Third Normal Form (3NF). Each table represents a single concept, non-key attributes depend on the primary key, and many-to-many data is separated into junction tables. Product tags are not stored as comma-separated strings but normalized into Tag and product_tags tables. Order items store purchased item snapshots to preserve order history, which is an intentional and justified denormalization for business correctness.

5.5 Indexing Strategy

- Primary keys are indexed by default.
- Foreign keys such as user_id, product_id, category_id, order_id, cart_id should be indexed for join performance.
- Frequently searched fields such as product name, username, email, category, and order status should have indexes.
- Composite indexes can be added for common queries such as product category + status or order customer + status.

6. Implementation Details

6.1 Authentication and Authorization

Furnituree implements token-based authentication using JWT. When a user logs in successfully, the backend returns a signed token containing the username and role-related information. The frontend stores this token and sends it in the Authorization header for protected API calls. Spring Security validates the token and checks role permissions before allowing sensitive operations.

Code snippet from `JwtUtil.java`:

```
public static String generateToken(String username) {

        return Jwts.builder()

                .setSubject(username)

                .setIssuedAt(new Date())

                .setExpiration(new Date(System.currentTimeMillis() +
EXPIRATION_TIME_MS))

                .signWith(key, SignatureAlgorithm.HS256)

                .compact();

    }
```

Code snippet from `SecurityConfig.java`

```
@Bean

    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{

        http

                .cors(cors ->
cors.configurationSource(corsConfigurationSource()))

                .csrf(csrf -> csrf.disable())

                .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))

                .authorizeHttpRequests(auth -> auth

                        // Preflight CORS
```

```
                        .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()


                        // Public auth APIs
                        .requestMatchers("/auth/**").permitAll()
.anyRequest().authenticated())
                .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class);


        return http.build();
```

JWT authentication is important because the system has protected features such as cart, checkout, order history, and admin management. After login, the backend generates a token. The frontend sends this token with later requests. The backend then validates the token and checks the user role. This separates normal users, managers, and admins.

6.2 Product Search, Filter, Sort, and Pagination

Purpose:
This logic allows users to search products by keyword, filter by category/price/stock, sort products, and use pagination.

Code snippet from `product_controller.java`:

```
@GetMapping("/search")
    public List<Product> search(@RequestParam String keyword) {
        return productService.search(keyword);
    }


    @GetMapping("/filter")
    public Page<Product> filterProducts(
            @RequestParam(required = false) String keyword,
            @RequestParam(required = false) Double minPrice,
            @RequestParam(required = false) Double maxPrice,
            @RequestParam(required = false) Long minQuantity,
            @RequestParam(required = false) Long maxQuantity,
```

```
            @RequestParam(required = false) Long categoryId,
            @RequestParam(defaultValue = "all") String stockStatus,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size,
            @RequestParam(defaultValue = "id") String sortBy,
            @RequestParam(defaultValue = "asc") String sortDir) {


        return productService.filterProducts(
                keyword, minPrice, maxPrice, minQuantity, maxQuantity,
categoryId, stockStatus,
                page, size, sortBy, sortDir);
    }
```

Code snippet from `ProductService.java`:

```
Sort sort = direction.equalsIgnoreCase("desc")
        ? Sort.by(sortBy).descending()
        : Sort.by(sortBy).ascending();
Pageable pageable = PageRequest.of(page, size, sort);
return productRepo.searchProducts(keyword, categoryId, minPrice, maxPrice, inStock, pageable);
```

This logic is more complex than a normal product list because it combines many conditions at the same time. Users can search by keyword, filter by category, filter by price range, show only in-stock products, sort results, and move between pages. This improves performance because the frontend does not need to load all products at once.

6.3 Checkout, Order Creation, and Stock Deduction

Code snippet from `OrderService.java`:

```
for (CartItem cartItem : cartItems) {
            Product product =
productRepo.findById(cartItem.getProduct().getId())
                    .orElseThrow(() -> new ResourceNotFoundException("Product
not found"));
```

```
            long requestedQuantity = cartItem.getQuantity();
            long stock = product.getQuantity() == null ? 0 :
product.getQuantity();

            if (requestedQuantity <= 0) {
                throw new BadRequestException("Invalid cart item quantity");
            }
            if (stock < requestedQuantity) {
                throw new BadRequestException("Not enough stock for product: "
+ product.getProduct_name());
            }

            double price = product.getPrice();
            double lineTotal = price * requestedQuantity;
            subtotal += lineTotal;

            product.setQuantity(stock - requestedQuantity);
            productRepo.save(product);

            OrderItem orderItem = new OrderItem();
            orderItem.setOrder(order);
            orderItem.setProduct(product);
            orderItem.setProductName(product.getProduct_name());
            orderItem.setQuantity(requestedQuantity);
            orderItem.setPrice(price);
            orderItem.setLineTotal(lineTotal);
            order.getOrderItems().add(orderItem);
```

```
        }

Payment payment = new Payment();

        payment.setOrder(order);

        payment.setMethod(request.getPaymentMethod());

        payment.setAmount(totalAmount);

        if (request.getPaymentMethod() == PaymentMethod.MOCK_CARD) {

            payment.setStatus(PaymentStatus.PAID);

            payment.setPaidAt(LocalDateTime.now());

        } else {

            payment.setStatus(PaymentStatus.UNPAID);

        }
```

Checkout is complex because it must follow a complete business workflow. The system first validates the cart and checks product stock. Then it creates an order, creates order items, creates a payment record, deducts stock, and clears the cart. This prevents overselling and ensures that order data remains consistent across multiple database tables.

6.4 Advanced Features Implemented

Advanced Feature	Implementation Summary
CSV Export	Admin export function for product/order/report data.
Dashboard Analytics	Summary metrics for products, users, orders, and business status.
Real-Time Chat	WebSocket-based customer-admin communication with admin notifications.

6.5 Business Logic

- Prevent adding unavailable or inactive products to cart.
- Prevent cart quantities from exceeding available inventory.
- Validate checkout only when the cart has valid items.
- Store order item price at purchase time to preserve historical order accuracy.
- Support order status workflow such as PENDING, CONFIRMED, SHIPPING, COMPLETED, or CANCELLED.
- Support review moderation before displaying customer feedback publicly.

6.6 Error Handling and Validation

The backend uses custom exceptions and centralized exception handling to return structured messages. The frontend avoids blindly parsing empty responses as JSON and displays meaningful messages for network errors, validation errors, and server errors. This improves usability and prevents confusing browser console errors during demo.

6.7 Security Implementation

The Furnitureee system implements several security mechanisms to protect user accounts, private API endpoints, and application data. The main security features include password hashing, SQL injection prevention, XSS prevention, CSRF handling, JWT authentication, and role-based authorization.

Password Hashing

The system does not store plain-text passwords in the database. Passwords are hashed before being saved.

user.setPassword(encoder.encode(user.getPassword()));

The project uses Spring Security password encoding, such as BCrypt. BCrypt is suitable for password hashing because it automatically uses salt and is intentionally slow, which makes brute-force attacks harder.

When a user logs in, the system compares the raw password with the hashed password:

encoder.matches(rawPassword, hashedPassword);

This protects user accounts even if the database is exposed.

SQL Injection Prevention

The project uses Spring Data JPA repositories instead of manually building SQL strings. Repository methods and parameter binding help prevent SQL injection.

Example:

userRepo.findByUsername(username);

productRepo.findById(id);

Because user input is passed as parameters instead of being directly concatenated into SQL queries, attackers cannot easily inject malicious SQL commands.

XSS Prevention

XSS can happen when user input is displayed directly on the page as HTML. To reduce this risk, the frontend should display dynamic data using safe text rendering instead of injecting raw HTML.

Recommended safe approach:

element.textContent = product.productName;

Avoid unsafe rendering:

element.innerHTML = userInput;

The project also validates form input such as checkout name, phone number, and address to reduce invalid or malicious input.

CSRF Protection

The backend uses JWT-based authentication for API requests. Since the frontend sends the JWT token manually in the `Authorization` header, the project does not rely on cookie-based session authentication.

Because of this, CSRF risk is lower than traditional session-cookie systems. In Spring Security, CSRF can be disabled for REST APIs:

.csrf(csrf -> csrf.disable())

This is acceptable for this project because authentication is handled using JWT tokens, not server-side sessions.

Authentication Mechanism

The project uses JWT authentication. After successful login, the backend generates a token and returns it to the frontend.

The frontend sends the token in protected API requests:

Authorization: Bearer <JWT_TOKEN>

The backend checks the token before allowing access to protected resources. This ensures that only logged-in users can access features such as cart, checkout, profile, and order history.

Authorization Checks

The system uses role-based authorization.Access control is based on role:

user → browse products, manage cart, checkout, view own orders

manager → manage products, categories, and orders

admin → full management access, including user management

This prevents normal users from accessing admin or manager functions. For example, only admin or manager accounts should update products, categories, and order statuses.

6.8 AI Assistance Report

During the Furnitureee E-commerce project, AI tools such as ChatGPT were used as learning and development support tools. AI was used to explain concepts, suggest implementation approaches, help debug errors, generate documentation drafts, and support diagram preparation. However, all AI suggestions were reviewed, tested, and modified to match the actual project requirements and codebase.

6.8.1 How AI Was Used

AI was mainly used for the following tasks:

- Explaining Spring Boot, REST API, JWT authentication, and role-based authorization.
- Suggesting how to organize the project using Controller-Service-Repository architecture.
- Helping design the database relationships and explain one-to-many and many-to-many relationships.

- Suggesting the Product–Tag many-to-many implementation using a `product_tags` junction table.
- Helping plan the checkout workflow, including order creation, payment record creation, stock deduction, and cart clearing.
- Helping debug errors from Spring Boot, MySQL, Git, and Railway deployment logs.
- Assisting with technical report sections, README, API documentation, ERD, Use Case Diagram, Architecture Diagram, and Data Flow Diagram.

AI was not used to blindly generate the whole project. Each suggestion was checked against the real source code, database tables, and project requirements.

6.8.2. Examples of AI-Assisted Components

One important AI-assisted component was the checkout workflow. AI helped outline the correct business process:

Validate checkout information

Check cart items

Validate product stock

Create order

Create order items

Create payment record

Deduct product stock

Clear cart items

This workflow was then adapted into the actual project code and tested through the frontend checkout page.

Another example was the Product–Tag many-to-many relationship. AI explained that one product can have many tags and one tag can belong to many products. Based on this explanation, the project used a junction table called `product_tags` to connect `product` and `tags`.

AI also helped with report diagrams. For example, the ERD and DFD were revised several times to match the real database. The final database contains 10 tables:

user, category, product, tags, product_tags,

cart, cart_item, orders, order_item, payment

The WebSocket chat feature was also clarified. Although the project supports real-time chat, messages are not stored in MySQL. Therefore, no `messages` table was included in the final ERD or table schema.

6.8.3. What Was Modified and Why

Not all AI outputs were used directly. Several suggestions had to be modified because they did not exactly match the real project.

For example, some AI-generated code snippets used method names or file names that were different from the actual project. These snippets were not copied directly. Instead, the real project files were checked before adding code snippets to the report.

The diagrams were also modified. Earlier versions included a `Messages` table for chat, but this was removed because the actual WebSocket chat only works in real time and does not persist data in the database.

The deployment configuration was also reviewed. Environment variables were used instead of hard-coded database passwords or JWT secrets. This made the project safer for GitHub and deployment.

6.8.4. Problems Found and Fixed

AI helped identify and debug several problems during development. One major issue was a Spring Boot deployment crash caused by duplicate `DataSeeder` classes. The error showed that `dto.DataSeeder` conflicted with `config.DataSeeder`. The duplicate file was removed, and only the correct seeder in the `config` package was kept.

Another issue was related to database and documentation consistency. The report originally mentioned a chat message table, but the actual MySQL database did not have one. This was corrected so that the report matched the real database.

AI also helped with Git commands, Railway deployment steps, `.env.example`, database scripts, and documentation structure.

6.8.5. What I Learned

Using AI helped me understand the project better instead of only copying code. I learned how to:

- Structure a Spring Boot project using Controller-Service-Repository layers.
- Use JWT authentication and role-based authorization.
- Design normalized database relationships.
- Implement a many-to-many relationship using a junction table.
- Build a realistic e-commerce checkout workflow.
- Read and debug Spring Boot error logs.
- Write technical documentation that matches the real project.

Overall, AI was used as a support tool for learning, debugging, and documentation. The final implementation was tested and adjusted by the team to make sure the project works correctly and can be explained during presentation.

8. Testing and Quality Assurance

8.1 Testing Strategy

Testing focused on end-to-end user flows, role-based access, database compatibility, cart behavior, checkout, and admin functions. Because this is a Web Application Development project, emphasis was placed on stable live demonstration and core feature correctness.

ID	Test Scenario	Input/Condition	Expected Result
TC01	Register new customer	Valid username/email/password	Account is created and customer can login.

TC02	Login as customer	customer / 123456	Token stored and customer pages are accessible.
TC03	Login as admin	admin / 123456	Admin dashboard is accessible.
TC04	Access admin page as customer	Customer token	Access is denied or redirected.
TC05	Search product	Keyword: sofa	Only matching products are displayed.
TC06	Filter empty category	Category: Poufs if no data exists	Empty state appears but reset/other categories still work.
TC07	Add item to cart	Available product	Cart badge increases and item appears in cart.
TC08	Remove one cart item	Cart with multiple products	Selected item is removed and others remain.
TC09	Checkout	Cart with valid items and delivery information	Order is created successfully.
TC10	Checkout empty cart	No items	System blocks checkout with clear message.
TC11	Admin product search	Keyword in admin page	Admin table filters matching products.
TC12	Admin logout	Admin clicks logout	Token is cleared and login page is shown.

8.2 Bugs Found and Fixes

Issue	Cause	Fix
Login/register returned 500	CORS wildcard was combined with allowCredentials(true), and controller-level @CrossOrigin(*) conflicted with global CORS.	Replaced wildcard origins with allowedOriginPatterns and removed controller-level wildcard origins.
Unexpected end of JSON input	Frontend tried to parse empty or non-JSON responses using response.json().	Added safer response parsing and better error handling.
Database schema mismatch	Old MySQL database lacked newer audit columns and role values.	Added migration compatibility and seeding improvements; Docker database can be reset cleanly.
Cart item remove message appeared but item remained	Cart object/list was not refreshed after deleting one CartItem.	Added direct delete query by cartId/productId and refreshed frontend cart state.
Poufs empty category stuck on No pieces found	Empty state was not reset when switching filters.	Improved frontend filter reset and rendering logic.

8.3 Known Limitations

Although the Furnitureee system works for the main e-commerce workflow, there are still some limitations.

Limitation 1: WebSocket Chat Is Not Persisted

The WebSocket chat works in real time, but chat messages are not stored in the database. If the page is refreshed, previous chat messages may be lost.

Reason:
The current implementation focuses on real-time communication only.

Future improvement:
Add a `messages` table to store chat history.

Limitation 2: Payment Is Simulated

The payment feature records payment method and status, but it does not connect to a real payment gateway.

Reason:
Real payment integration requires third-party services, security verification, and additional configuration.

Future improvement:
Integrate a real payment provider such as Stripe, PayPal, or VNPay.

Limitation 3: Deployment Depends on External Platform Setup

The project can run locally, but public deployment requires correct environment variables and database configuration.

Reason:
Backend, database, and frontend deployment depend on services such as Railway, Netlify, or Vercel.

Future improvement:
Complete a stable deployment guide and provide a public URL for demo.

9. Deployment and Operations

9.1 Local Deployment

The application can run locally using Maven and a local MySQL database. The recommended local command is:

```
mvnw.cmd clean spring-boot:run
```

The application is then accessible at http://localhost:8080. Test accounts are seeded for admin, manager, and customer roles.

9.2 Docker Deployment

Docker Compose is used to run the Spring Boot application and MySQL database together. Docker makes the environment more consistent across machines, but the local machine must have Docker Desktop running and available ports configured correctly.

```
docker compose down -v
docker compose up --build
```

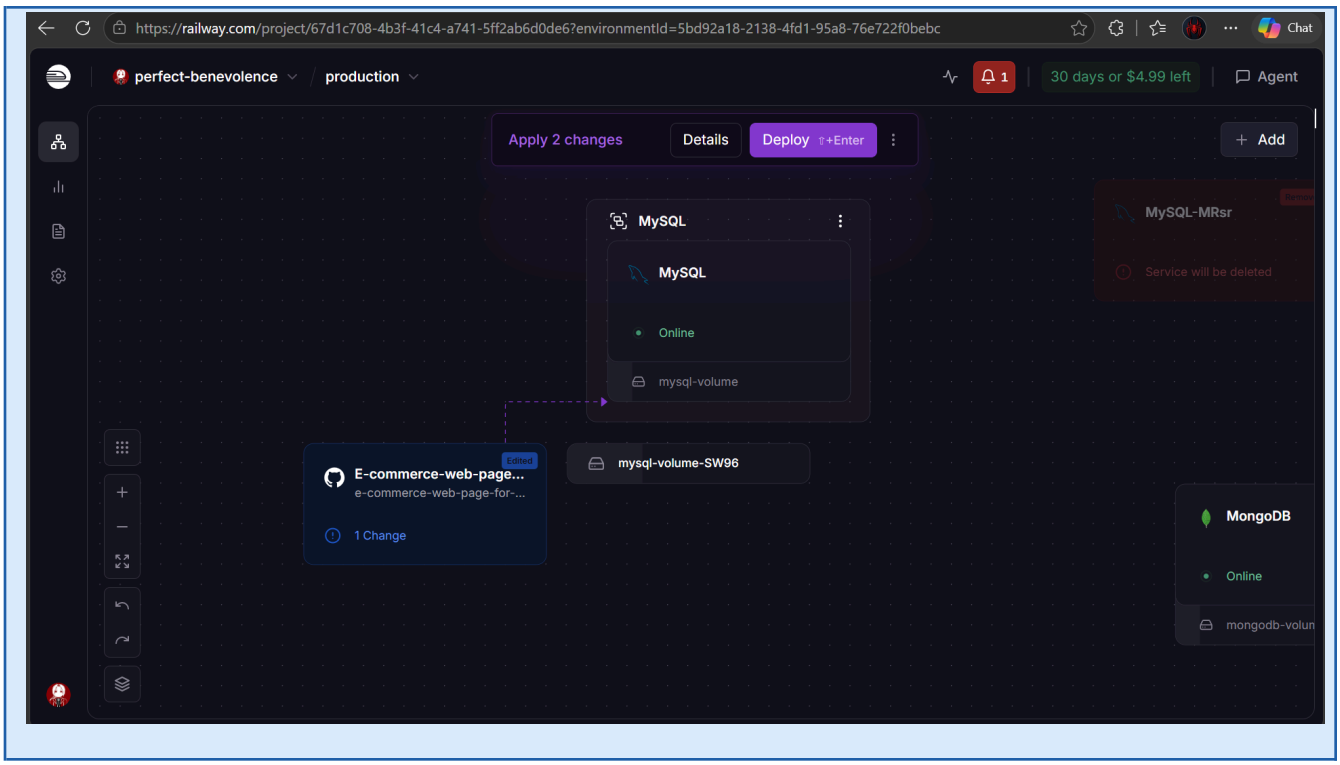
9.3 Environment Configuration

Variable/Setting	Example	Purpose
SPRING_DATASOURCE_URL	jdbc:mysql://mysql:3306/furniture_store	Database connection string inside Docker network.
SPRING_DATASOURCE_USERNAME	root	Database username.
SPRING_DATASOURCE_PASSWORD	[configured password]	Database password.
SERVER_PORT	8080	Backend web server port.
JWT_SECRET	[secure secret in production]	Secret key for signing JWT tokens.

9.4 Public Deployment Requirement

For final submission, the application should be deployed to a public hosting platform such as Render, Railway, Fly.io, or a VPS. The final report should include the live URL and test credentials. If online deployment is not completed, the team should clearly demonstrate local and Docker execution during presentation.

Screenshot: Deployment Proof


https://railway.com/project/67d1c708-4b3f-41c4-a741-5ff2ab6d0de6?environmentId=5bd92a18-2138-4fd1-95a8-76e722f0bebc
Chat
perfect-benevolence
production
1
30 days or $4.99 left
Agent
Apply 2 changes
Details
Deploy ⇧+Enter
Add
MySQL
MySQL
Online
mysql-volume
MySQL-MRsr
Service will be deleted
E-commerce-web-page...
e-commerce-web-page-for-...
Edited
1 Change
mysql-volume-SW96
MongoDB
Online
mongodb-volun

10. User Guide

10.1 Test Accounts

Role	Username	Password	Purpose
Admin	admin	123456	Access full admin dashboard and management functions.
Manager	manager	123456	Access management/dashboard features.
Customer	customer	123456	Shop, cart, checkout,, and chat.

10.2 Feature Walkthroughs

Login

Users enter their username and password on the Login page. After successful login, the system redirects them based on their role.

User → product catalog, cart, checkout, my orders

Manager → product, category, and order management

Admin → full management access, including user management

Cart

Users can view selected products in the cart, update quantity, remove items, and check the total price.

Steps:

1. Open Cart page.

2. Update or remove cart items.

3. Confirm cart total before checkout.

Product Catalog

Users can browse products, view product details, search by keyword, filter by category or price, sort products, and use pagination.

Steps:

1. Open product catalog.

2. Search or filter products.

3. View product details.

4. Add selected product to cart.

Checkout

Checkout creates an order from the cart.

Steps:

1. Add products to cart.

2. Open Checkout page.

3. Enter full name, phone number, address, city, and payment method.

4. Submit checkout form.

After checkout, the system creates an order, order items, payment record, deducts product stock, and clears the cart.

My Orders

Users can view their order history and order status from the My Orders page.

Common order statuses:

PENDING

CONFIRMED

SHIPPING

COMPLETED

CANCELLED

Users can cancel eligible orders if the order is still in an allowed status.

Admin / Manager Features

Admin and manager users can manage products, categories, and orders.

Main functions:

- Add, edit, or delete products

- Manage categories

- View all orders

- Update order status

- View dashboard information

Admin can also manage users, including editing user information and deactivating accounts.

WebSocket Chat

The system supports real-time chat between users and admin/manager.

Steps:

1. Open the system in two browser tabs.

2. Login as user in one tab.

3. Login as admin or manager in another tab.

4. Send a message.

5. The message appears instantly without refreshing.

Chat messages are real-time only and are not stored in the database.

Tips

- Use the user account to test shopping workflow.

- Use admin or manager accounts to test management pages.

- Check My Orders after checkout to confirm order creation.

- Use MySQL Workbench to verify stock, order_item, and payment records.

- Use two browser tabs to test WebSocket chat.

11. Team Collaboration

11.1 Team Organization

The team consists of two members. Work was divided by feature ownership, but both members contributed to integration, testing, and final documentation. The following contribution table is a suggested distribution and should be updated if the real workload differs.

Member	Main Responsibilities	Estimated Contribution
Nguyen Phuoc Thinh	Backend integration, database design, authentication/RBAC, cart/checkout, report coordination, final testing.	50%

Tran Le Minh Quan	Frontend UI pages, product catalog styling, admin page UI, screenshots, documentation support, presentation preparation, testing.	50%

11.2 Collaboration Method

- Tasks were divided by module and merged after local testing.
- Project issues were tracked informally during implementation and through debugging sessions.
- Git is recommended for final submission to show meaningful commit history and individual contribution.
- Before presentation, both members should rehearse the demo and prepare answers for technical questions.

11.3 Suggested Git Commit Strategy

- feat(auth): add JWT login and register
- feat(product): implement product CRUD and search
- feat(cart): add cart badge and remove item
- feat(order): add checkout and order workflow
- feat(chat): add real-time chat notification
- docs(report): add final technical report
- fix(cors): correct credentials and allowed origins

13. Conclusion and Future Enhancements

13.1 Project Summary

Furnituree successfully demonstrates a realistic full-stack web application for a furniture e-commerce domain. The project includes customer shopping flow, admin management, authentication and authorization, database relationships, business logic, real-time chat, dashboard support, documentation, and Docker execution. It is appropriate for a Web Application Development final project because it shows integration across frontend, backend, database, security, and deployment concerns.

13.2 Lessons Learned

- Full-stack integration requires consistent API routes, data models, and frontend expectations.
- Authentication and CORS must be configured carefully to avoid hidden runtime errors.
- Database schema changes can break existing local data unless migration is planned.
- Docker simplifies deployment but requires understanding of ports, volumes, containers, and database initialization.
- A stable demo is more important than adding too many unfinished features.

13.3 Future Enhancements

- Add a customer order history page with order detail view.
- Add admin order status update workflow with notification to customer.
- Add payment gateway integration such as VNPay, Stripe, or PayPal for real transactions.
- Improve product recommendation based on category, tags, or user behavior.
- Add automated tests for services and controllers.
- Deploy the application to a stable public URL for final grading and user testing.

13.4 Individual Reflection

During this project, I learned how to build a more complete web application instead of only creating simple CRUD pages. I improved my understanding of Spring Boot, MySQL, REST APIs, JWT security, and frontend-backend integration.

One important thing I learned was how to debug problems by reading error messages carefully. For example, deployment and database errors required checking logs, environment variables, file structure, and database configuration. This helped me become more confident in solving technical issues.

If I could do the project differently, I would plan the database and API structure earlier before implementing the frontend pages. Some features had to be adjusted because the database or backend logic changed during development. I would also write more automated tests to reduce manual testing time.

Overall, this project helped me grow as a developer. I became better at connecting frontend and backend code, designing database relationships, writing documentation, and explaining technical decisions. I also learned that building a real web application requires both coding skills and careful planning.

References

Spring Boot Documentation. https://docs.spring.io/spring-boot/

Spring Security Reference. https://docs.spring.io/spring-security/reference/

Spring Data JPA Reference. https://docs.spring.io/spring-data/jpa/reference/

Hibernate ORM Documentation. https://hibernate.org/orm/documentation/

MySQL Documentation. https://dev.mysql.com/doc/

Docker Documentation. https://docs.docker.com/

OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/

MDN Web Docs - JavaScript, HTML, CSS, Fetch API. https://developer.mozilla.org/

JWT RFC 7519. https://www.rfc-editor.org/rfc/rfc7519

BCrypt Password Hashing Concept. https://auth0.com/blog/hashing-in-action-understanding-bcrypt/

Appendices

Appendix A - Diagram Checklist

Diagram/Screenshot	Recommended Content
Use Case Diagram	Actors: Guest, Customer, Admin, Manager. Use cases: login, browse, cart, checkout, manage products, manage users, chat.
ER Diagram	All main tables with PK/FK and cardinality labels.
Architecture Diagram	Browser, REST API, WebSocket, service layer, repository layer, MySQL.
Sequence Diagram - Checkout	Customer -> frontend -> order controller -> order service -> cart/order/payment repositories -> database.
Sequence Diagram - Chat	Customer/Admin -> WebSocket -> chat controller -> messaging template -> subscribers.
Screenshots	Home, login, product search, cart, checkout, admin dashboard, chat, Docker running.

Appendix B - Demo Script

1. Open home page and explain Furnituree problem domain.

2. Show responsive layout and product catalog search/category filter.

3. Login as customer and add product to cart; show cart badge.
4. Open cart, remove one item, proceed to checkout, and place order.
5. Login as admin, search product, show dashboard and logout.
6. Demonstrate customer-admin chat and admin notification.
7. Briefly show database/ERD and architecture diagram.
8. Explain security: JWT, BCrypt, RBAC, validation, and CORS fix.
9. Mention Docker support and deployment plan.

Appendix C - Expected Q&A Preparation

Question	Suggested Answer
Why Spring Boot?	It provides a production-style framework with REST controllers, dependency injection, validation, security, and JPA support.
Why MySQL?	The e-commerce domain uses structured relationships that fit relational modeling and SQL constraints.
How is security handled?	BCrypt for passwords, JWT for authentication, and RBAC for admin/manager/customer authorization.
What is the most complex feature?	Checkout and cart/order workflow, plus real-time chat notification using WebSocket.
What would you improve next?	Deploy online, add order history/status updates, add automated tests, and improve CI/CD.

Appendix D - Final Submission Checklist

- Replace all placeholder names, IDs, URLs, and dates.
- Insert real screenshots and diagrams into the marked placeholder areas.
- Update Table of Contents after inserting images: Ctrl + A -> F9 in Microsoft Word.
- Push source code to GitHub with meaningful commits.
- Deploy the application or prepare a reliable local/Docker demo.
- Prepare test accounts and sample data before presentation.
- Rehearse the 20-minute presentation and live demo.